

SSAO Assignment Guideline

2n session: Implementing a basic SSAO

In this second session, we will focus on the second step of our rendering pipeline to create the first version of the SSAO estimation. You can download PLY models to test your technique from anywhere.

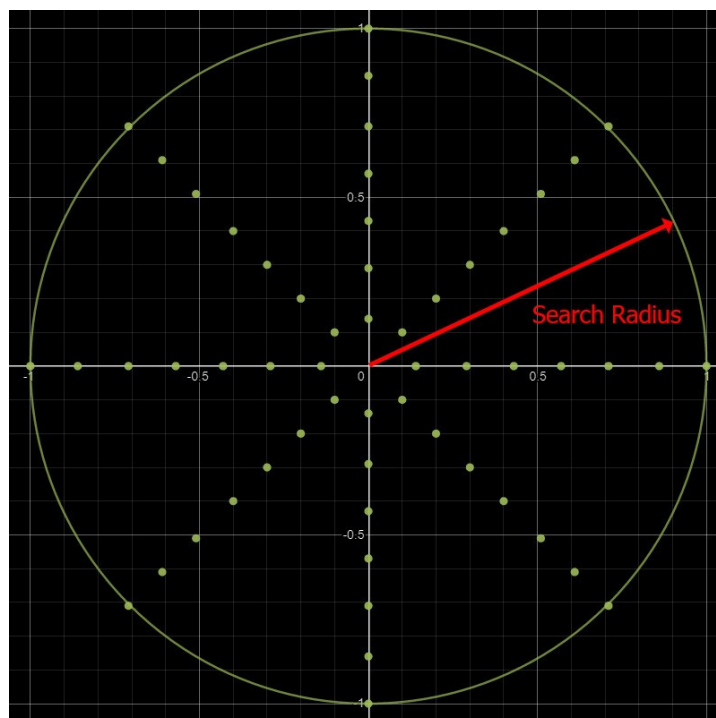
Sampling the data

In the introduction, we already explained that we will estimate the AO terms for all the pixels of the viewport. However, instead of considering the geometry of the model, we will only focus on the available data: our normal and depth buffers stored in images.

The idea of this approximation is that, for each pixel, neighboring geometry will be located in the neighboring pixels. So, we will estimate the occlusion of the surroundings by sampling the neighboring pixels. Obviously, we will only work on the fragment shader.

In this assignment, I ask you to implement a simple kernel. **You can implement any simple kernel**, but I propose a circular kernel to sample the neighboring pixels. Instead of sampling inside a matrix, we will sample the surrounding pixels in equidistant directions. For each direction, we will sample it in equidistant positions. We will let the user define the number of directions and samples per direction using the GUI. Moreover, we will need to consider how far we would like to sample. For that, we will also let the user define the radius of this circular sampling.

The following image illustrates our kernel



Take into account that, in any kernel, fragments near the border of the viewport, some samples will lie outside the bounds. You will need to handle them.

Recovering position from depth map

To recover a 3D position of the pixel from the depth map, we need to understand how perspective projection works. We can not directly use the depth information, because it is not linear: we have more precision close to the near plane than close to the far plane:

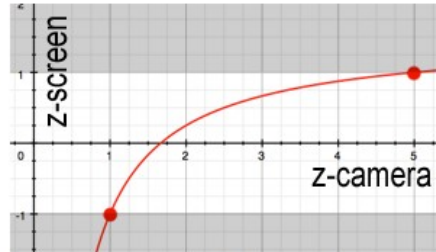
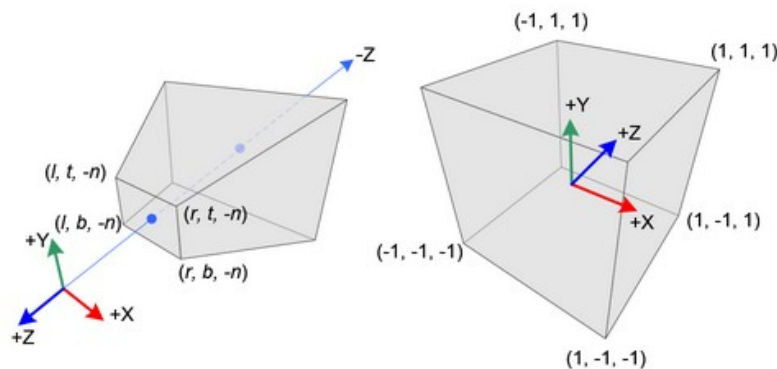


Figure 3: the remapping of the projected point's z coordinate is non linear. This graph shows the result of P_{8z} for near = 1 and far = 5.

src: www.scratchapixel.com

Before starting to transform the depth value, it is important to understand how perspective projection and NDC works:



Perspective Frustum and Normalized Device Coordinates (NDC)

src: <http://www.songho.ca>

The projection matrix is defined as:

$$\begin{bmatrix} \frac{2n}{r-l} & 0 & \frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & \frac{t+b}{t-b} & 0 \\ 0 & 0 & -\frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

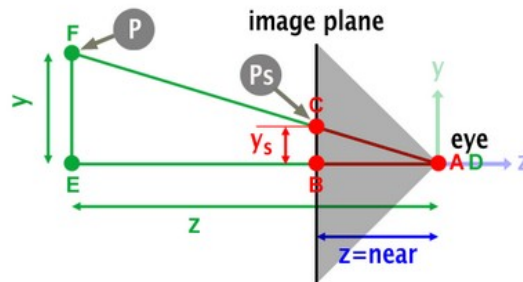
and the transformation from eye space to NDC:

$$\begin{pmatrix} x_{clip} \\ y_{clip} \\ z_{clip} \\ w_{clip} \end{pmatrix} = M_{projection} \cdot \begin{pmatrix} x_{eye} \\ y_{eye} \\ z_{eye} \\ w_{eye} \end{pmatrix}, \quad \begin{pmatrix} x_{ndc} \\ y_{ndc} \\ z_{ndc} \end{pmatrix} = \begin{pmatrix} x_{clip}/w_{clip} \\ y_{clip}/w_{clip} \\ z_{clip}/w_{clip} \end{pmatrix}$$

So, to transform the depth to Pz in eye space, we need to transform depth to NDC and then make the inverse transformation from NDC to clip space and then to eye space:

```
z_ndc = 2.0 * depth - 1.0;
z_eye = 2.0 * n * f / (f + n - z_ndc * (f - n));
```

Once, we know the z in eye space we can easily compute x and y in eye space. The trick is to use similar triangles:



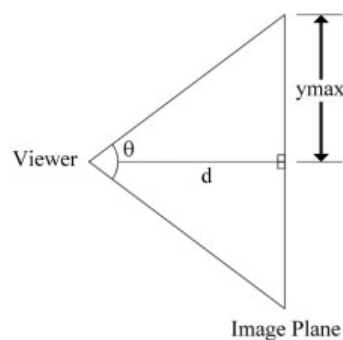
src: www.scratchapixel.com

The triangle ABC is similar to the triangle DEF. That means:

$$\frac{AB}{DE} = \frac{BC}{EF}.$$

So $EF = BC * DE / AB$ where $EF = x_eye$, $DE = z_eye$, $AB = z_near$ and we can compute BC using the texCoords of the fragment (transformed from $[0,1]$ to $[-1,1]$) multiplied by the width of the image plane. This works in a similar way to $EF = y_eye$, but using the height of the image plane instead of the width.

We can compute the width and the height of the plane using basic trigonometry taking into account the fov angle and aspect ratio:



Estimating the ambient occlusion term

Once we have recovered the position and normal of each of the samples, we can estimate the ambient occlusion contribution of each of them. However, remember we are performing a very approximated version of a Montecarlo integration. So remember that you need to sum all the contributions and normalize them to obtain a final value inside $[0,1]$.

After estimating AO, we will need to visualize it. This is quite straightforward, you only need to take into account that we want to visualize the inverse of AO to render darker the more occluded parts.